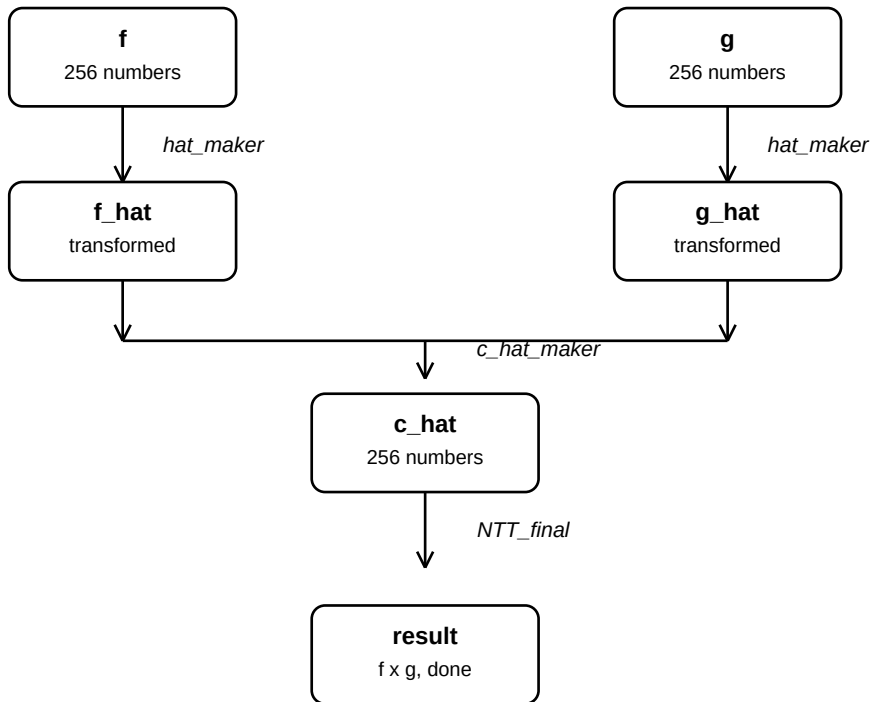


NTT Multiplication - Full Writeup

Real Kyber scale: $n=256$, $q=3329$, $\text{zeta}=17$. Everything below is confirmed, hand-checked, and verified against the slow schoolbook method (MATCH: True).

Part 1: The Maths

The whole pipeline, as a diagram



The single formula used in hat_maker (forward)

Take a pair (a, b) and this group's zeta:

```
t = zeta * b mod q
new a = (a + t) mod q
new b = (a - t) mod q
```

The 7 rounds of hat_maker

Round	Groups	Zetas used	Pairing distance
1	1	1 (index 1)	128 apart
2	2	2 (indices 2-3)	64 apart
3	4	4 (indices 4-7)	32 apart
4	8	8 (indices 8-15)	16 apart
5	16	16 (indices 16-31)	8 apart
6	32	32 (indices 32-63)	4 apart
7	64	64 (indices 64-127)	2 apart

$1+2+4+8+16+32+64 = 127$ - every zeta gets used exactly once. Position 0 always lands in the first group each round, so it happens to use indices 1, 2, 4, 8, 16, 32, 64.

Real worked example - position 0, all 7 rounds of hat_maker

```
Round 1: (2619, 2044), zeta=1729 -> becomes 1297
Round 2: (1297, 1532), zeta=2580 -> becomes 2334
Round 3: (2334, 366), zeta=2642 -> becomes 567
Round 4: (567, 1293), zeta=1062 -> becomes 2185
Round 5: (2185, 3293), zeta=296 -> becomes 1516
Round 6: (1516, 3007), zeta=289 -> becomes 1670
Round 7: (1670, 1557), zeta=17 -> becomes 1507
```

The formula used in c_hat_maker (the multiply step)

Pairs here are consecutive neighbours (position 0&1, 2&3, ...) - NOT far apart like hat_maker's pairs. Every single pair gets its own gamma (no sharing within a group).

$$\begin{aligned}c_0 &= (a_0 \times b_0) + (a_1 \times b_1 \times \gamma) \mod q \\c_1 &= (a_0 \times b_1) + (a_1 \times b_0) \mod q\end{aligned}$$

Real worked example - pair 0 of c_hat_maker

$a_0, a_1 = 1507, 1456$ (from f_hat) $b_0, b_1 = 194, 1315$ (from g_hat) $\gamma = 17$

step 1: $a_0 \times b_0 = 1507 \times 194 = 292,358$
step 2: $a_1 \times b_1 = 1456 \times 1315 = 1,914,640$
step 3: $step2 \times \gamma = 1,914,640 \times 17 = 32,548,880$
step 4: $step1 + step3 = 292,358 + 32,548,880 = 32,841,238$
step 5: $\mod q \rightarrow c_0 = 653$

step 6: $a_0 \times b_1 = 1507 \times 1315 = 1,981,705$
step 7: $a_1 \times b_0 = 1456 \times 194 = 282,464$
step 8: $step6 + step7 = 1,981,705 + 282,464 = 2,264,169$
step 9: $\mod q \rightarrow c_1 = 449$

Result for pair 0: (653, 449). This repeats for all 128 pairs, filling c_hat (256 numbers).

The reverse process - NTT_final

Same butterfly idea as hat_maker, but undone:

- rounds go the OPPOSITE order: 2 apart, then 4, 8, 16, 32, 64, 128 apart
- reads zeta_reverse forward (zeta flipped end to end) instead of zeta backward
- the formula itself flips: add first, THEN multiply

```
new a = (a + b) mod q  
new b = zeta_reverse_value x (b - a) mod q
```

One final step after all 7 reverse rounds: multiply every number in the list by 3303 - this cancels out a leftover scaling factor left behind by doing 7 rounds. After that, the list is the real, final answer to $f \times g$.

Part 2: The Code

Everything above, as actual working Python, real names throughout:

zeta, zeta_reverse, gamma - the lookup tables

zeta: the 128 magic numbers straight from FIPS 203 Appendix A. Not calculated - just given to us, same idea as CBD_TABLE was.

zeta_reverse: literally zeta flipped end to end, so NTT_final can walk forward through it instead of walking zeta backwards.

gamma: a SEPARATE table of 128 numbers, also from Appendix A, used only in c_hat_maker, one per pair.

hat_maker(f) - turns a normal poly into its transformed shape

```
def hat_maker(f):
    f_hat = f.copy()
    i = 1
    length = 128
    while length >= 2:
        start = 0
        while start < n:
            z = zeta[i]
            i += 1
            for j in range(start, start + length):
                t = (z * f_hat[j + length]) % q
                f_hat[j + length] = (f_hat[j] - t) % q
                f_hat[j] = (f_hat[j] + t) % q
            start += 2 * length
        length //= 2
    return f_hat
```

c_hat_coefficient and c_hat_maker - the multiply step

```
def c_hat_coefficient(a0, a1, b0, b1, gamma_value):
    c0 = (a0 * b0 + a1 * b1 * gamma_value) % q
    c1 = (a0 * b1 + a1 * b0) % q
    return c0, c1

def c_hat_maker(f_hat, g_hat):
    c_hat = [0] * n
    for i in range(128):
        c0, c1 = c_hat_coefficient(
            f_hat[2*i], f_hat[2*i+1],
            g_hat[2*i], g_hat[2*i+1], gamma[i])
        c_hat[2*i] = c0
        c_hat[2*i+1] = c1
    return c_hat
```

NTT_final(c_hat) - turns c_hat back into the real answer

```
def NTT_final(c_hat):
    f = c_hat.copy()
    i = 0
    length = 2
    while length <= 128:
        start = 0
        while start < n:
            z = zeta_reverse[i]
            i += 1
            for j in range(start, start + length):
                t = f[j]
                f[j] = (t + f[j + length]) % q
                f[j + length] = (z * (f[j + length] - t)) % q
            start += 2 * length
        length *= 2
    f = [(x * 3303) % q for x in f]
    return f
```

Naming reference

Old name	New name
zetas	zeta
(new)	zeta_reverse
zetas_mult	gamma
NTT(f)	hat_maker
NTT_inverse	NTT_final
base_case_multiply	c_hat_coefficient
multiply_ntts	c_hat_maker
poly_mult_schoolbook	unchanged - unrelated to NTT itself

Final check

f (first 5): [2619, 456, 102, 3037, 1126]
g (first 5): [262, 1384, 86, 2409, 2268]
f_hat (first 5): [1507, 1456, 1833, 934, 1178]
g_hat (first 5): [194, 1315, 2820, 3001, 1536]
c_hat (first 5): [653, 449, 551, 1966, 576]
schoolbook result: [1331, 2055, 1467, 538, 3010]
NTT result: [1331, 2055, 1467, 538, 3010]

MATCH: True